Lifeboat: Rethink, Unique Ideas and Demo Plan

What to change after the mentoring feedback, what to build to stand out, and how to make the demo convincing.

1. What the judges told us

- This is a **data disaster** problem, not a natural calamity. Stop grouping it under floods and earthquakes.
- Example given: one provider with servers in Mumbai, Pune, Bangalore and Delhi. If one server or its data dies, traffic should move to another and the customer should not notice.
- Replication is not instant. A cart change made in Mumbai reaches the other regions after some time. We must show this honestly in the demo.
- Our presentation had no innovative or unique features.

2. Reframe the project (no engine change needed)

New story: **a multi-region data platform that survives a data disaster**. Disasters we cover: a region outage, a corrupted disk, a cut link, and a traffic spike. The customer should not notice any of them.

- Use a **shopping cart / orders** domain for the demo (the judges used the Amazon example themselves). Reports become cart items and orders.
- Run **four regions**: Mumbai, Pune, Bangalore, Delhi. The code already supports any number of peers; add one more process (Bangalore, port 8004).
- Everything built so far stays: event log, replication by watermark, partition switch, conflict detection, restart safety, roles, slow links.

3. The judge's point, and our answer

The judge is right. Asynchronous replication always leaves a gap between 'saved in Mumbai' and 'present in Delhi'. Every serious system has it. Do not hide it. **Make the gap visible, measured and controlled.** That is our differentiator: most teams will say 'it replicates instantly'.

What to say: "We chose availability, so every region always accepts writes. Because replication is asynchronous, we measure the gap, show how much data is at risk, send critical writes first, and give each customer a session guarantee so their own data never disappears."

4. Unique ideas, ranked

Effort figures are rough estimates for one person.

#	Idea	What it is	Why it is unique	Already have	Effort
1	**Data-at-risk meter + Kill Region scoreboard**	Live count: 'N acknowledged writes exist only here, oldest X s'. A real Kill Region button, then a scoreboard: writes lost, writes safe, time to recover.	Shows what a failure would actually cost (RPO). Few teams quantify loss.	/v1/status has nullable *risk* and *queues* fields; peer views; push queues.	3-4 h
2	**Session guarantee (read-your-writes)**	Each response carries a small session token (what you have seen). After failover the new region waits briefly until it holds that data, or marks the response 'catching up'.	Directly solves 'customer sees an empty cart after failover'.	Per-origin watermarks (contiguous) already track what each region holds.	4-5 h
3	**Failover gateway + idempotency keys**	One entry URL routes to the nearest region and silently reroutes if it dies. Idempotency keys stop a retried 'add to cart' or 'place order' from happening twice.	Zero visible interruption and no duplicate orders.	Idempotency table exists in the schema; admin and health endpoints.	4 h
4	**Replication timeline per write**	Realistic link delays (e.g. 1 s, 2 s, 4 s) and a race strip per write: Mumbai ok -> Pune ok 1.0 s -> Bangalore ok 2.1 s -> Delhi pending. Plus a 'time to consistent' stopwatch.	Turns the judge's concern into the best part of the demo.	Slow link mode with delay_ms and rate_bps; received_at stored per event.	2-3 h
5	**Self-healing from data corruption**	A 'Corrupt this region's disk' button damages stored rows. The region detects the hash mismatch, quarantines the bad data and re-fetches clean copies from peers.	A true data disaster, not just an outage. Repairs itself.	Every event is hashed; quarantine table; pull anti-entropy.	3-4 h
6	**Merge by data type**	Cart items merge as a union, quantities as counters, order status goes to review. Stretch: bounded stock (escrow) so the last item can never be oversold across regions.	Concurrent shopping is not a 'conflict'; no overselling.	Per-field policy design in the main plan; derive() shape unchanged.	6 h+
7	**Resilience report card**	After any scenario, auto-generate: data lost (RPO), time to recover (RTO), lost writes, duplicates, convergence time, verdict.	Proof instead of claims; easy for judges to compare.	Digests and state hashes; idea 1 supplies the numbers.	2 h
8	**Graceful degradation levels**	Browsing and add-to-cart always work. Checkout/payment needs at least one other region to confirm, otherwise it shows 'pending'.	Matches the PS title literally: graceful degradation.	CONSTRAINED and RECOVERING levels exist in the contract.	4 h

5. Recommended build set

Pick this set. It tells one coherent story and reuses what exists.

Order	Feature	One-line pitch for the judges
1	Replication timeline with realistic link delays (idea 4)	"We show the gap instead of hiding it."
2	Data-at-risk meter and Kill Region scoreboard (idea 1)	"We can tell you exactly what a failure would cost."
3	Session token + failover gateway + idempotency (ideas 2 and 3)	"The customer never sees their cart go backwards."

Order	Feature	One-line pitch for the judges
4	Corruption self-heal (idea 5)	"A data disaster, not just an outage, and it repairs itself."
5	Resilience report card (idea 7)	"Proof, not claims."
Stretch	Merge by type, bounded stock (idea 6), degradation levels (idea 8)	Only if all tests above are green.

6. How to fix the demo problem (nothing visible on one laptop)

On one machine replication takes milliseconds, so there is nothing to see. Fix it with these four things:

- **Realistic link delays.** Set per-link delays (Mumbai to Pune 1 s, Bangalore 2 s, Delhi 4 s) with the existing slow-link mode.
- **Four region panels** side by side, each polling every 500 ms, so judges watch the item appear one region at a time.
- **Per-write timeline strip** with a countdown to each region, and a final 'consistent in N s' stamp.
- **One-click scenarios**: 'Normal day', 'Black Friday on a bad network', 'Region dies mid-checkout', 'Disk corruption'.

7. Three-minute demo story

Step	Action	What the judge sees / what we say
1	Four regions, a customer adds items	The timeline shows each item crawling Mumbai, Pune, Bangalore, Delhi. "Replication takes a few seconds."
2	Point at the risk meter	"Right now 3 acknowledged writes exist only in Mumbai. That is the data at risk."
3	Press Kill Mumbai	The customer's next click is routed to Pune with no error. The cart is intact and nothing is duplicated.
4	Read the scoreboard	"0 acknowledged writes lost; 1 write was in flight and recovered."
5	Press Corrupt Delhi	It detects the damage, quarantines it and repairs itself from peers.
6	Open the report card	All regions converged with identical digests. "Proof, not claims."

8. Likely judge questions and short answers

Question	Short answer
Why not make it instant and strongly consistent?	Then a cut-off region would have to refuse customers. We chose availability and made staleness visible and bounded.
What if the region dies before it replicated?	Those unreplicated writes are lost. We measure exactly how many, shrink the window by sending critical writes first, and show it live.
How does the customer not notice failover?	A gateway reroutes, idempotency keys prevent duplicates, and a session token makes the new region wait for the customer's own data.
How do you know regions end up identical?	State is a pure function of the event set. We compare digests and state hashes, and test permutations, duplicates and partitions.
What is actually new here?	We quantify and show the replication gap and data at risk, give a session guarantee across failover, and self-heal corruption.

9. Honest scope: what exists vs what is new

Already working (verified, 162 tests passing)	To build for this pivot
Independent regions with own database; local write acknowledged after commit; push and pull replication by watermark; partition switch in both directions; slow link; conflict detection; restart safety; roles and history; status and digests; dashboard.	Fourth region; cart/orders domain labels; link delay presets; risk and queues numbers; Kill Region and Corrupt Region controls; session token and failover gateway; replication timeline UI; report card. Stretch: merge by type and bounded stock.

Not yet built and must not be described as finished: the review queue and resolve flow, signed events, the metrics API, and the randomized convergence test.

10. Decisions needed from the team

- **Domain:** switch the demo to a shopping cart / orders, or keep incident reports with new wording?
- **Regions:** move to four (Mumbai, Pune, Bangalore, Delhi) for the demo?
- **Scope:** confirm the recommended set in section 5, in that order, with stretch items only if everything else is green.
- **Lanes:** Rahil: risk and data-at-risk numbers, corruption detection. Simrit: link delay presets, Kill Region, scenarios. Akshit: session token, failover gateway, idempotency. Miti: timeline strip, four-region view, report card.